

Smoothed Analysis of Algorithms

Andreas Panayi
A.A.MA.

Alexandra Gypari
E.M.E.

Dept. Electrical and Computer Engineering
National Technical University of Athens

June 2025

Contents

- 1 Introduction
 - Establishing the context
 - Motive
- 2 Formal Definitions
- 3 Applications of Smoothed Analysis
 - Problem 1: Simplex Algorithm
 - Problem 2: Traveling Salesman Problem
- 4 Discussion
 - Open Problems
- 5 Bibliography
 - References
 - Thanks

Part 1:

INTRODUCTION

Non Formal Introduction

- *Smoothed analysis*¹, which is a “hybrid” analysis framework, blending average-case and worst-case analysis (and also perhaps the most well studied such “hybrid”).
- *Smoothed analysis* is a way to explain why some algorithms work well in practice even if they seem slow in theory.

¹Due to lack of formal translation of the term into Greek, a few suggestions are:

,

Δ

The Cons of Worst-Case Analysis

- Overly pessimistic performance predictions.
- Can rank algorithms inaccurately.
- No data model.

The Cons of Average-Case Analysis

- Strong dependence on the input distribution
- Unrealistic symmetry assumptions
- May miss adversarial structure

Smoothed Analysis: Bridging the Gap

- Start with an input
- Add a bit of noise
- Average performance
- Take the worst such performance

Part 2:

FORMAL DEFINITIONS

Smoothed Analysis Definition

Consider a cost measure $\text{COST}(A, z)$ for algorithms A and inputs z . Usually COST is the running time of A on z .

For a fixed input size n and measure σ of perturbation magnitude, standard definition of the *smoothed complexity* of A is

$$\sup_{z \text{ of size } n} \mathbb{E}_{r(\sigma)}[\text{COST}(A, z + r(\sigma))].$$

An algorithm has polynomial smoothed complexity if its smoothed complexity is a polynomial function of both n and $1/\sigma$.

Asymptotic Behavior

We observe that:

- As the perturbation size σ goes to 0, this measure tends to worst-case analysis.
- As the perturbation size goes to infinity (the perturbation dwarfs the original input) and the measure tends to average-case analysis (under the perturbation distribution).

Part 3:

APPLICATIONS OF SMOOTHED ANALYSIS

Simplex Algorithm

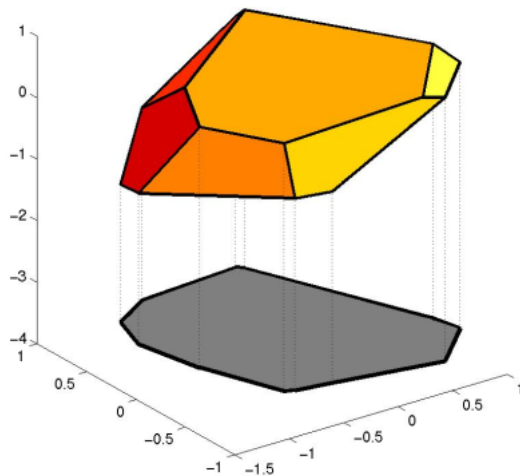
- ① The simplex algorithm for linear programming, effectively does greedy local search along the “edges” of the feasible region, hopping from one vertex to another.
- ② The number of vertices of the feasible region can be exponential in the number of dimensions (i.e., variables), as in e.g. the hypercube.
- ③ So, the Simplex Method has exponential worst-case running time (as shown by the Klee-Minty “squashed hypercube” [[KM72](#)])
- ④ Although it is almost always very fast (typically near-linear) in practice.

The first “killer application” of smoothed analysis is the result that the simplex algorithm has polynomial smoothed complexity [[ST03](#)].

Proof Sketch

- The pivot rule (that dictates which polytope vertex to go to next) used was the “shadow pivot rule”.
- The idea is to project the high-dimensional feasible region onto a plane (the “shadow”), where running the simplex algorithm is easy.
- It can be arranged so that the optimal solution appears as a vertex in the shadow.
- The key result in [ST03], is that *the expected number* of vertices of the shadow (over the randomness in the perturbations) is polynomial in the input size and $1/\sigma$.

Proof Sketch



Perturbation Model

The perturbation model is: independently for each entry of the constraint matrix and right-hand side of the linear program, a Gaussian (i.e., normal) random variable is added, with mean 0 and standard deviation σ .

Proof Plan

The proof plan in many smoothed analyses, including that in [ST03], follows a two-step approach:

1. First, one identifies a sufficient condition on the data under which the given algorithm has good performance.
2. Second, one shows that the sufficient condition is likely to hold for perturbed data.

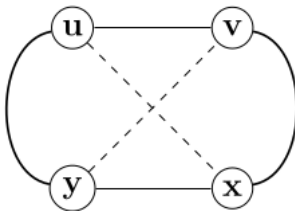
Approach For The Simplex Method

- With each vertex v of the shadow we can associate an angle of $\pi - \delta_v$ radians, for some $\delta_v > 0$.
- If you go around the boundary of the shadow, finishing up where you started, the sum of the δ_v 's is 2π .
- If $\delta_v \geq \gamma$ for every v , the number of vertices of the shadow is at most $2\pi/\gamma$.
- Now we have to show that γ is very likely (over the perturbations) to be at least inverse polynomial in the input size and $1/\sigma$.

Smoothed Analysis of Local Search for TSP

- Consider the 2-OPT Heuristic for TSP in the plane with the l_1 metric
- Given n points on the unit square define a tour $x_1, \dots, x_n$ in order to minimise $\sum_{i=1}^n \|x_{i+1} - x_i\|_1$
- Allowed moves: Swaps (NB There are $O(n^2)$ possible swaps in a given tour)
- A swap is called improving if it yields a tour with smaller "length".
- Worst case: Algorithm might require exponential number of iterations before reaching locally optimal solution.

Smoothed Analysis of Local Search for TSP



Smoothed Analysis of Local Search for TSP

- Pick density function f_i from which each x_i is drawn
- Assumption 1: $f_i(\mathbf{x}) \leq \frac{1}{\sigma}$ for all i and x in the unit square (so that $\text{supp}(f)$ has area at least σ)
- Assumption 2: All points lie in the unit square with probability 1.
- Call a swap ϵ -bad if the improvement it yields lies in $(0, \epsilon)$.
- Removing the edges $(\mathbf{u}, \mathbf{v})$, $(\mathbf{x}, \mathbf{y})$ and replacing them with edges $(\mathbf{u}, \mathbf{x})$, $(\mathbf{v}, \mathbf{y})$ yields improvement $|u_1 - v_1| + |u_2 - v_2| + |x_1 - y_1| + |x_2 - y_2| - |u_1 - y_1| - |u_2 - y_2| - |v_1 - x_1| - |v_2 - x_2|$

Smoothed Analysis of Local Search for TSP

- ① Theorem: The smoothed complexity of the 2-OPT heuristic for the TSP in the plane equipped with the l_1 metric is polynomial. In particular, the expected number of iterations is $O(\sigma^{-1}n^6 \log(n))$
- ② Lemma: For every perturbed instance and $\epsilon > 0$ the probability that there is an ϵ -bad swap is $O(\epsilon n^4 / \sigma)$

Smoothed Analysis of Local Search for TSP

- Smoothed complexity theorem for 2-OPT heuristic is an immediate consequence of the lemma
- Any tour has length $\leq 2n$ and if there are no ϵ -bad swaps the algorithm terminates within $\frac{2n}{\epsilon}$ iterations
- Each iteration must improve the length of the tour so that $\# \text{iterations} \leq n!$
- Using the lemma with $\epsilon = \frac{2n}{M}$ and the fact that $\ln(n!) = O(n \log(n))$ we have:

Smoothed Analysis of Local Search for TSP

$$\begin{aligned}\mathbb{E}[\text{\#iterations}] &= \sum_{n=1}^{n!} \mathbb{P}[\text{\#iterations} \geq M] \\ &\leq \sum_{n=1}^{n!} \mathbb{P}[\exists \frac{2n}{M}\text{-bad swap}] \\ &\leq \sum_{n=1}^{n!} \frac{n^5}{M\sigma} \\ &= O(\sigma^{-1}n^6 \log(n)) \quad \square\end{aligned}$$

Smoothed Analysis of Local Search for TSP

- For the proof of lemma note that there are only $O(n^4)$ fundamentally distinct swaps so we need to show a bound of $O(\frac{\epsilon}{n})$ on the probability of a given swap being bad
- The improvement $|u_1 - v_1| + |u_2 - v_2| + |x_1 - y_1| + |x_2 - y_2| - |u_1 - y_1| - |u_2 - y_2| - |v_1 - x_1| - |v_2 - x_2|$ lies in $(0, \epsilon)$ only if one of the $(4!)^2$ distinct linear combinations (of the 8 variables) lies in $(0, \epsilon)$.
- For each fixed linear combination the probability that it lies in $(0, \epsilon)$ is at most $\frac{\epsilon}{2\sigma}$
- Immediate consequence of the $\frac{1}{\sigma}$ bound for the pdfs f_i and the fact that the range of values for a fixed variable that cause the linear combination to lie in $(0, \epsilon)$ is an interval of length at most $\frac{\epsilon}{2}$ $\square$

Part 4:

DISCUSSION

Open Problems

① Sparsity-preserving perturbations

- Special structure of some sparse matrices in network-flow problems is not preserved since the perturbation method is applied to zero entries of the constraint matrix, thus altering the sparsity.

② Maximum Cut problem with the “flip” neighborhood

- We don't know if local search has polynomial smoothed complexity for this problem in general, although we know this is true for complete graphs or graphs with maximum degree $O(\log(|V|))$.

Part 5:

BIBLIOGRAPHY

References I

- [KM72] Victor Klee and George J. Minty. “How Good is the Simplex Algorithm?” In: *Inequalities III*. Ed. by O. Shisha. New York: Academic Press Inc., 1972, pp. 159–175.
- [Rou20] Tim Roughgarden. “Beyond the Worst-Case Analysis of Algorithms (Introduction)”. In: *arXiv preprint arXiv:2007.13241* (2020). Version 1, 26 Jul 2020.
- [ST03] Daniel A. Spielman and Shang-Hua Teng. “Smoothed Analysis of Algorithms: Why the Simplex Algorithm Usually Takes Polynomial Time”. In: *arXiv preprint arXiv:cs/0111050* (2003). Version 7, 9 Oct 2003. MIT, Boston University, Akamai Technologies Inc.

References II

- [ST08] Daniel A. Spielman and Shang-Hua Teng. “Smoothed Analysis: An Attempt to Explain the Behavior of Algorithms in Practice”. In: *Communications of the ACM* 51.10 (2008). Yale University and Boston University, pp. 101–107.

Thanks

Thanks for your attention ☺